

FIAREP — Backend Wiring Specification

Purpose: Complete reference for building the server API and migrating the app off local-first SQLite. Every table, entity, endpoint, role rule, and cross-device concern is documented here so nothing is lost when the backend is wired.

Current state: The iOS app (React Native / Expo SDK 54, expo-router, bundle `com.tmstni.construction-pm`) is fully local-first. All data lives in on-device SQLite (`construction.db`) via `lib/store.ts`. Each table is stored as (`id TEXT PRIMARY KEY, state TEXT`) where `state` is JSON of the typed entity (a few use (`key/projectId TEXT PRIMARY KEY, state TEXT`)). Nothing syncs across devices today. The backend's job is to become the source of truth and let multiple devices (CPM, Supervisor, Procurement, Inspector, Vendor) share data.

Backend repo today: `ssh -i ~/.ssh/fiarep.pem ubuntu@18.219.209.89, ~/fiarep-api`, Express 5 + Drizzle + RDS Postgres, branch `master`. Auth/developments/staff invites/tenant isolation already scaffolded there.

1. Roles & identity

AppMode (who is using the device): `resident | administrator | management | worker | inspector | vendor | procurement`.

StaffRole (issued login accounts): `administrator | management | worker | inspector | procurement`. Vendors have NO login — they're identified by a job tracking ID (`sr-XXXX`) only. Residents have no login.

StaffPosition (job title on a staff account): Plumber, Electrician, Inspector, CPM, Staff Worker, Elevator Service, Superintendent, Maintenance Worker, Carpenter, Roofer, General Construction, CCTV Installation, Heating Service, Caretaker, Other.

Key role/position rules the backend must enforce (currently UI-gated only): - **CPM vs Inspector** share `inspector` mode but differ by `position`. CPM sees Submit Scope + Scope of Work (Divisions); Inspector sees Log Violations + My Routes. A CPM has position `CPM`; an inspector has position `Inspector`. - **Procurement** is its own login. Only procurement may broadcast, award, delete, rate-and-close, and return-to-supervisor on procurement records. Management and Administrator are READ-ONLY on procurement. - **Closed procurement jobs** are immutable: approve/return/reject/broadcast must be rejected server-side once status = `closed`. - **Scope pricing visibility:** Inspectors never see pricing (room lines, elevator costs, estimate totals). Vendors never see the CPM's internal prices — they get the scope with prices stripped and only fill their own Unit Cost. - **Admin issues** administrator/management/procurement accounts; management issues worker/inspector accounts.

Auth today: role PINs (`getRolePin / setRolePin`), per-staff codes (`verifyStaffLogin, resetStaffCode`), remembered staff per role. Replace with real auth (JWT/session) server-side; keep the issued-code concept for staff onboarding.

2. Tables → entities

Each row below is a current SQLite table and the endpoints it needs. All are per-tenant (authority) scoped — the backend already has tenant isolation.

staff_accounts (type: StaffAccount)

Fields: `id, name, firstName, lastName, position (StaffPosition), role (StaffRole), code, status (StaffStatus: pending/approved/denied/refused/revoked), developments[], requestedAt, issuedBy`. Endpoints: `list (by status), request, approve, deny, refuse, revoke, issue, issueFull, resetCode, delete, verifyLogin, bulk-add`. Functions: `requestStaffAccount, approveStaffAccount, denyStaffAccount, refuseStaffAccount, revokeStaffAccount, issueStaffAccount, issueStaffAccountFull, resetStaffCode, deleteStaffAccount, verifyStaffLogin, addBulkPendingEmployees, listStaffAccounts, listStaffByPosition, listAssignableByTrade, listFieldInspectors, listManagementForDevelopment, hasAnyAdministrator, hasApprovedManagement, bootstrapAdministrator, developmentsForManager`.

projects (type: Project)

Fields: `id, name, client, rates (custom rate table or null), meta (owner, assigned inspector, review state, approved flag)`. Endpoints: `list, get, create, delete, setRates, assignInspector, submitForReview, setReview, approve-lock`. Functions: `listProjects, getProject, createProject, deleteProject, setProjectRates, assignProjectInspector, getProjectInspector, getProjectOwner, submitProjectForReview, setProjectReview, getProjectReview, isProjectApproved, listApprovedProjectIds`.

rooms (type: Room) — child of projects

Fields: `id, projectId, name, unit, lines[] (LinItem: category/description/qty/ unitPrice/origPrice/priceBy), photos[], walls2d[], scan`. Endpoints: `list by project, add, update, delete`. Functions: `listRooms, addRoom, getRoom, updateRoom, deleteRoom`. NOTE: photos are local file paths today → must move to S3 (see §5).

cost_estimates (keyed by projectId) — the "Nature of Work" internal estimate

State: `CostEstimateState` (header, rows keyed by category, totals). Functions: `getCostEstimate, setCostEstimate`.

project_scopes (keyed by projectId) — the HPD Division/Section scope form

State: `VendorScope` (header + `divisions[]→sections[]→lines[] .desc/qty/unit/unitCost`). This is the CPM's internal scope WITH prices. Functions: `getProjectScopeForm, setProjectScopeForm`.

checklists / inspections / intakes / elevators / roofplans (keyed by projectId)

Per-project sub-documents (renovation checklist, building inspection, intake report, elevator services w/ costs, roof plan sketch). Functions: get/set each. Elevator + estimate carry pricing → inspector view must strip (see §1).

procurement (type: ProcurementRequest)

The scope→bid→award lifecycle. Fields: id, trackingId (sr-XXXXX, blank until broadcast), projectId, address, scope, scopeFile/scopeFileName, status (ProcurementStatus: draft/submitted/approved/pending/bidding/awarded/closed), requestedBy, requestedAt, approvedBy/At, invitedAt, vendor, awardedBy/At, closedAt, performance, amountCharged, deduction, finalAmount, returnNote, returnedAt. Lifecycle: CPM creates draft (submitProjectScope / createProcurementRequest) → submitScopeForApproval (→ management) → approveProcurementRequest (→ procurement) → broadcastProcurement (generates trackingId, → vendors) → submitBid (vendors) → awardProcurementRequest → rateAndCloseProcurement (closed). Return paths: rejectScope (mgmt→CPM), returnScopeToManagement (procurement→mgmt). Guards: closed = immutable. Only procurement acts. Functions: createProcurementRequest, updateScopeDraft, submitScopeForApproval, approveProcurementRequest, rejectScope, returnScopeToManagement, broadcastProcurement, getProcurementRequest, getProcurementByTracking, listProcurementRequests, awardProcurementRequest, closeProcurementRequest, rateAndCloseProcurement, deleteProcurementRequest, submitProjectScope, listReturnedScopes.

procurement_bids (type: ProcurementBid) — child of procurement

Fields: id, requestId, trackingId, vendorName, amount, note, submittedAt. Each vendor company files a separate bid on the same trackingId. Functions: submitBid, listBids, submitVendorQuoteAsBid.

vendor_quotes (keyed by trackingId::vendorName)

State: VendorScope. The outward form the vendor fills — loaded from the CPM's project_scope with prices stripped and work locked; vendor sets unitCost only. Grand total → bid. Functions: getVendorQuote, setVendorQuote, submitVendorQuoteAsBid.

vendor_contacts (type: VendorContact)

Fields: id, name, phone, email. Functions: addVendorContact, listVendorContacts, updateVendorContact, deleteVendorContact.

building_violations (type: BuildingViolation) — inspector-logged HPD violations

Fields: id, building, violationNo, code, codeDesc, hazardClass (A/B/C), notes, loggedBy, loggedAt. Functions: addBuildingViolation, listBuildingViolations, deleteBuildingViolation. (Code catalog is static in lib/violationCodes.ts, 383 codes, 341 with full official text/abstract/hint — no table needed, ship as reference data.)

route_assignments (type: RouteAssignment) — supervisor→inspector address lists

Fields: id, inspector, assignedBy, assignedAt, fileName, stops[] (RouteStop: id/address/status pending|reached|not_reached). "Done for the day" bumps pending→ not_reached and sorts unreached to top. Functions: createRouteAssignment, listRouteAssignments, getRouteAssignment, setRouteStopStatus, finishRouteDay.

violations (type: ViolationLookup) — supervisor→CPM/inspector "go look this up"

Fields: id, violationNumber, address, unit, note, sentTo, sentBy, sentAt, acknowledgedAt. Functions: sendViolationLookup, listViolationLookups, getViolationLookup, acknowledgeViolationLookup, deleteViolationLookup.

resident_reports (type: ResidentReport) — resident-submitted issues

Fields: id, location, unit, development, description, status, updates[] (ResidentUpdate), assignedTo, ratings. Functions: createResidentReport, listResidentReports, getResidentReport, updateResidentReportStatus, addResidentUpdate, assignResidentReport, rateAndResolveReport, deleteResidentReport, findResidentReports, findReportByRef, sendReportToAdmin, createManagementReport, setReportDevelopment.

change_orders (type: ChangeOrder) — mid-job change requests

Fields: id, reportRef, description, status (ChangeOrderStatus), createdByName, recipient, verdict, reason. Functions: createChangeOrder, listChangeOrders, listChangeOrdersForRecipient, respondToChangeOrder, deleteChangeOrder.

priority_violations (type: PriorityViolation) — red/urgent inspection findings

Routed to assignee + supervisors; escalates over time. Functions: raisePriorityViolation, listPriorityViolations, outstandingPriorityFor, acknowledgePriorityViolation, supervisorsForPriority.

notifications (type: Notification) — the inbox

Fields: id, target (role bucket OR staff name), message, detail, read, at, reportId (optional; prefixes: proj : , hud : , else procurement/report id). Tap routing is by reportId prefix + message text. Functions: addNotification, listNotifications, listAllNotifications, unreadCount, markNotificationRead, markNotificationsRead, deleteNotification, removeNotificationsByRef. BACKEND: this is the key real-time surface → needs push (APNs) + per-user delivery.

project_notes / project_reviews (children of projects)

Notes thread + supervisor Approve/Needs-revision/Reject decisions.

audit_log (type: AuditEntry)

Who did what. Functions: logAudit, listAuditLog, deleteAuditEntry, clearAuditLog.

settings (key/value)

Global rates, current actor, remembered staff, app mode, role PINs. Functions: getGlobalRates, setGlobalRates, getAppMode, setAppMode, clearAppMode, getCurrentActor, setCurrentActor, clearCurrentActor, getRememberedStaff, setRememberedStaff, clearRememberedStaff, getRolePin, setRolePin, getStaffPin, setStaffPin. NOTE: actor/mode/remembered-staff are DEVICE-LOCAL — keep on device, do NOT move to server. Global rates + role PINs DO move server-side.

contractor scores / vendor scores (computed, no table)

getContractorScores, getVendorScores, ContractorScore, VendorScore, VendorPerformance. Vendor score = 60% performance + 25% on-time (14d) – 15% deduction rate. Compute server-side from procurement history.

3. Suggested REST surface

Group by resource. All under /api/v1, tenant-scoped via auth token.

- POST /auth/login (staff code or role PIN) → token
- GET/POST /staff, POST /staff/:id/{approve,deny,revoke,reset-code}, DELETE /staff/:id
- GET/POST /projects, GET/DELETE /projects/:id, POST /projects/:id/{rates,assign,submit-review,review}
- GET/POST /projects/:id/rooms, PUT/DELETE /rooms/:id
- GET/PUT /projects/:id/{estimate,scope,checklist,inspection,intake,elevator,roofplan}
- GET/POST /procurement, POST /procurement/:id/{submit,approve,reject,return,broadcast,award,rate-close}, DELETE /procurement/:id
- GET /procurement/by-tracking/:trackingId (vendor lookup, no auth — the ID is the key)
- GET/POST /procurement/:id/bids
- GET/PUT /vendor-quotes/:trackingId/vendor, POST .../submit
- GET/POST /vendor-contacts, PUT/DELETE /vendor-contacts/:id
- GET/POST /building-violations, DELETE /building-violations/:id
- GET/POST /routes, POST /routes/:id/{stop-status,finish-day}
- GET/POST /violation-lookups, POST /:id/ack, DELETE /:id
- GET/POST /resident-reports (+ status, updates, assign, resolve, delete)
- GET/POST /change-orders, POST /:id/respond, DELETE /:id
- GET/POST /priority-violations, POST /:id/ack
- GET /notifications (per user), POST /:id/read, POST /read-all, DELETE /:id
- GET /audit-log, GET/PUT /settings/{rates,role-pins} (global only)
- GET /scores/{vendor,contractor} (computed)

4. Notification & routing model (real-time)

Notifications target either a ROLE BUCKET (management, administrator, procurement) or a STAFF NAME. The inbox reads both (role bucket + own name). For the backend: - Resolve role-bucket notifications to every user currently in that role. - Deliver via APNs push + a GET /notifications pull. - Preserve the reportId prefix convention (proj: , hud:) — the app routes taps on it. New procurement/violation refs should adopt explicit prefixes too (currently text-matched, which is brittle — good time to fix server-side). - removeNotificationsByRef clears stale ones (e.g. superseded scope approvals) — keep this dedupe behavior server-side.

APNs is the single biggest missing piece — every cross-actor handoff (scope submitted, returned, bid received, route assigned, violation to look up, priority finding) is a notification. Without push, users must open the app to see them.

5. Files & photos (S3)

Local-only today, cannot cross devices: - Room photos (rooms.photos[]), scans, roof plan sketches. - Scope attachment files (procurement.scopeFile) via lib/files.ts → documentDirectory + files/. - Generated PDFs/Excel (estimate report, scope Excel) — ephemeral, share-sheet only; no need to store unless you want a record. Plan: upload to S3, store the S3 key in place of the local path. The upload button already exists; wire it to a presigned-URL flow. CloudFront/S3 deploy infra is already in use for the app.

6. Migration path (local SQLite → server)

1. Stand up the REST surface above; keep the JSON-blob storage model server-side if you want a fast port (a state jsonb column per table mirrors the app), OR normalize into real columns using the types as the schema. Recommend normalize for procurement, bids, staff, projects, rooms (queried/filtered); JSON blob is fine for the per-project sub-documents (estimate, scope, checklist, elevator, etc.).
2. Add a thin data layer in the app that calls the API and falls back to / caches in SQLite for offline. lib/store.ts is the single choke point — every function above is already the seam. Reimplement each to hit the API.
3. Keep DEVICE-LOCAL: current actor, app mode, remembered staff (settings table subset). Everything else becomes server-authoritative.

4. Wipe test data (tenant `test-authority`) before connecting — still PENDING.
5. Enforce all \$1 role rules server-side (they're UI-only today).

7. Known backlog / gotchas to carry over

- iOS branch `main` , backend branch `master` — inconsistent, reconcile.
- Drizzle journal not regenerated after manual 0001/0002 migrations.
- Session invalidation on tier/role change (not done).
- City violation lookup (HPD/DOB via NYC Open Data / Socrata by BBL/address) — never built, backend-only. Would auto-fill `building_violations` from real data.
- Critical alerts entitlement for priority/urgent findings (APNs).
- Cross-device sync is the whole point — nothing shares today.
- Vendor `sr-` IDs are delivered out-of-band (no in-app channel to vendors yet).
- 383 HPD violation codes live in `lib/violationCodes.ts` as static reference — ship as seed data, not user-editable.
- 35 CSI divisions / 133 sections in `lib/vendorScope.ts` — same, static reference.

Generated from the live codebase inventory (25 tables, 34 types, ~150 store functions). `lib/store.ts` is the single seam between the app and future backend — every function listed maps to an endpoint or a computed/local concern above.